732A94 Advanced R programming

Lecture 2

Johan Alenlöv (Slides based on Leif Jonssons, Måns Magnusson and Krzystof Bartoszek)

Linköpings Universitet

Lecture 2:

- Program control
- Functions
- Environments and scoping
- Random number generators
- Function arguments
- Returning values
- Specials
- Functionals
- Functional programming
- R packages

Questions?

Any questions from last time?

Program control

Two main ways to control program flow

- Conditional statements
- Loops

Conditional statements

```
if(Boolean expression) {
  # Statements
} else if (Boolean expression) {
  # Statements
} else {
  # Statements
}
```

Brackets `{...}`:

- Not needed if single line follows `if`, `else if` or `else`
- but defensive programming

Loops

Three types:

- `for` (entry controlled loop)
- `while` (entry controlled loop)
- `repeat` (exit controlled loop)

Brackets `{...}`:

- Not needed if single line follows `for`, `while`
- but defensive programming

For loop

```
for(name in vector) {
  # Statements
}
```

```
x <- c("a","d")
for(i in 1:length(x)) {
  print(x[i])
}
```

```
## [1] "a"
## [1] "d"
```

```
x <- c()
for(i in 1:length(x)) {
  print(x[i])
}
```

```
## NULL
```

seq_along

Use when looping over vectors

```
x <- c("a","d")
for(i in seq_along(x)) {
  print(x[i])
}
```

```
## [1] "a"
## [1] "d"
```

```
x <- c()
for(i in seq_along(x)) {
  print(x[i])
}
```

While loop

```
while (Boolean expression) {
  # Statements
}
```

```
x <- 5
while(x > 1) {
  print(x)
  x <- x %/% 2
}
```

```
## [1] 5
## [1] 2
```

Controlling loops

- `break` (loop)
- `next` (iteration)

```
for(i in 1:10) {
  if(i %% 2 == 0) {
    next
  } else if (i > 6) {
    break
  }
  print(i)
}
```

```
## [1] 1
## [1] 3
## [1] 5
```

Repeat loop

```
repeat {
  # Statements

  # break
}
```

- repeat needs `break` statement
- brackets `{...}` are needed
- high risk of infinte loop

Functions revisited

```
my_function_name <- function(x, y){
  z <- x**2 + y**2
  return(z)
}
```

Function components

A function consists of

- Function arguments
- Function body
- Function environment

These can be accessed using

- `formals(f)`
- `body(f)`
- `environment(f)`

Built in function editor - `fix(f)`

Lexical scoping

How does R find stuff?

Goes in order:

- Current environment
- Parent environment
- ⋮
- Global environment
- ... along searchpath to ...
- Empty environment (fail)

Enivronment search path

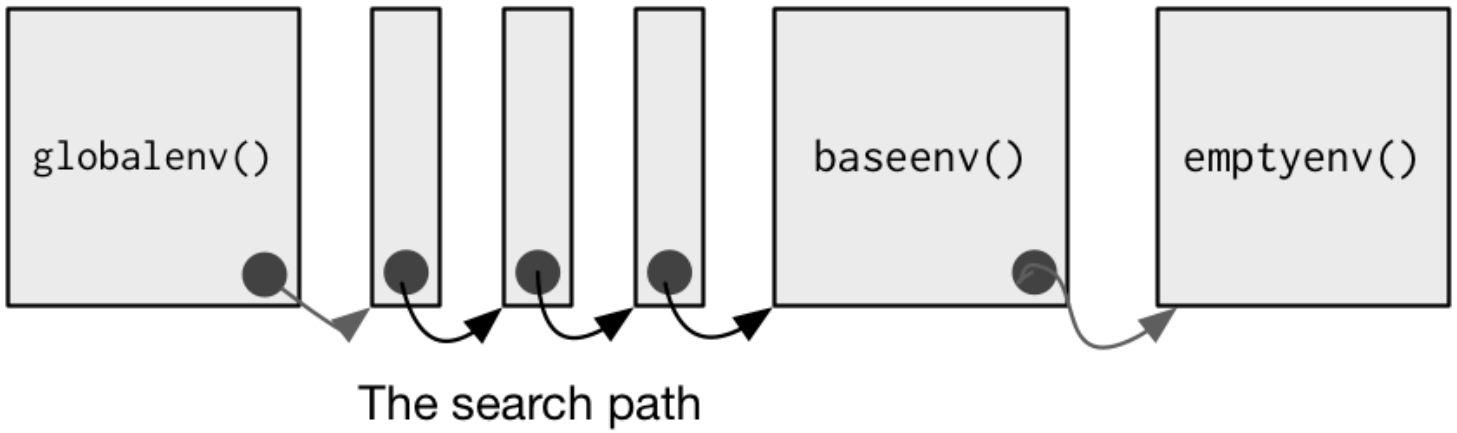

Figure 1: Environment serach-path (H. Wickham, Adv. R)

```
search()
```

```
## [1] ".GlobalEnv"        "package:stats"     "package:graphics"
## [4] "package:grDevices" "package:utils"     "package:datasets"
## [7] "package:methods"   "Autoloads"         "package:base"
```

Environments

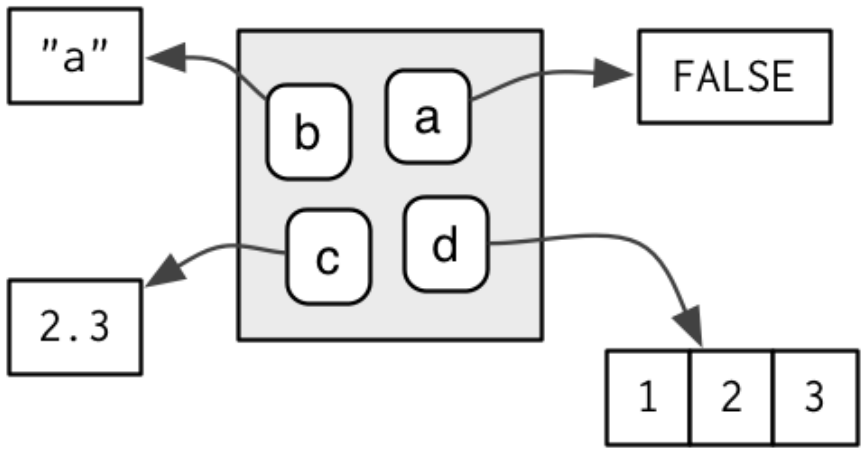

Figure 2: Environment is a bag of names (H. Wickham, ADv. R)

Environments

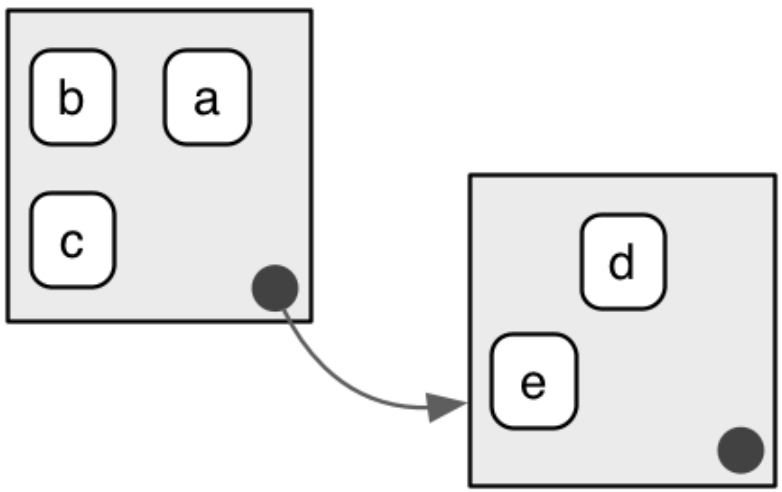

Figure 3: Environments has a parent but no children (H. Wickham, Adv. R)

Working with environments

```
ls()
```

```
## [1] "e"
```

```
ls(e)
```

```
## [1] "a" "b" "c" "d"
```

```
ls(all.names = TRUE) # Shows hidden
```

```
## [1] "e"
```

Working with environments

Removing variables

```
rm()
rm(list = ls()) # Delete all

gc() # Invoke garbage collector (free memory)
```

Assignments

- `<-` Shallow assignment Inside current environment
- `<<-` Deep assignment Inside Parental environment, if not found then global environment
- `assign()`. Full control assignment (**not** recommended) Can set strange variable names

```
assign("x/x",3);ls();get("x/x")
```

```
## [1] "x/x"
```

```
## [1] 3
```

Assignments

- = Shallow assignment, equivalent to <- in the most **external scope** In function calls they have **different** meaning:
  - <- assigns values
  - = defines parameter values

```
mean(x = 1:100)
```

```
## [1] 50.5
```

```
mean(y = 1:100)
```

```
## Error in `mean.default()`:
## ! argument "x" is missing, with no default
```

```
mean(y <- 1:100)
```

```
## [1] 50.5
```

Lazy evaluation

Variables assigned values only when needed

```
f1 <- function(x){globalVar <<- 3; x}
f2 <- function(x){x <- x + 0; globalVar <<- 3; x}
# Are f1() and f2() the same?
```

```
globalVar <- 2
f1(globalVar)
```

```
## [1] 3
```

```
globalVar <- 2
f2(globalVar)
```

```
## [1] 2
```

Random number generators

Required for simulations – provide numbers mimicking a given random variables.

Theory will be discussed in Computational Statistics course

Functions: `rDistributionName`, e.g. `rnorm()`, `rexp()`

Related functions:

- `dDistributionName` pdf / pmf
- `pDistributionName` cdf
- `qDistributionName` quantile
- `sample` used for sampling from a vector

Random seeds

When using pseudo-random numbers we set a seed, this defines what random numbers will be generated and allows for reproducible results when using random numbers.

- `set.seed(value)` or `.Random.seed <- value`

```
set.seed(1)
rnorm(1)
```

```
## [1] -0.6264538
```

```
rnorm(1)
```

```
## [1] 0.1836433
```

```
set.seed(1)
rnorm(1)
```

```
## [1] -0.6264538
```

Setting up the generator

Different generator types (algorithms) Can specify generator for Gaussian and discrete unform. pseudo-random numbers.

See `?RNGkind` for more information (default is fine!)

Use `RNGversion` to set the algorithm to how they were in earlier versions of R

Assignments: be careful

The object `.Random.seed` is only looked for in the user's workspace.

See `732A94_AdvancedRHT2026_Lecture02_Slide25.R`

Function arguments

copy-on-modify semantics

- “modifying a function argument does not change the original value”

Specify arguments by:

- Position
- complete name
- partial name

```
my_function_name(1, 2)
my_function_name(firstarg = 1, secondarg = 2)
my_function_name(f = 1, s = 2)
```

Just in case: partial names cannot be used in function body!

Function arguments

copy-on-modify semantics

- `do.call()`
- `missing()`

```r
do.call(my_function_name, list(1,2))
do.call("my_function_name", list(f=1, s=2))
```

`missing()` check if argument is missing in function

Return values

- The last expression evaluated in a function
- Return multiple values using a list
- `return()` used to specify what is returned.
- `on.exit()` is called when the function exits and is used to clean up.

Pure function: “Pure functions map the same input to the same output and have no other impact on the workspace. In other words pure functions have no side effects, they don’t affect the state of the world in any way apart from the value they return” (H. Wickham, Adv. R)

Specials

Prefix functions:

- Most functions in R, the name comes before the arguments

Infix functions:

- Functions that comes between the arguments:
  - `+`, `-`

Replacement functions:

- Functions that act like they modify the arguments, but in reality create a copy.
  - Usually `function_name<-`

Infix functions

- Useful to define arithmetic operations
- Examples: `+`, `&`, `<-`, `%*%`, `%/%`

```
x <- 1
x + 1
```

```
## [1] 2
```

```
"+"(x,1)
```

```
## [1] 2
```

```
"<-"(x,3)
x
```

```
## [1] 3
```

Infix functions

- User defined infix functions must start and end with **%**
- Name of function has to be but in **backticks** when defining

```
"%+%" <- function(a, b) paste(a,b)
"new" %+% "string"
```

```
## [1] "new string"
```

Replacement functions

- When defining replacement function's name has to be put in **backticks**
- Typically 2 arguments: object to modify and with what to modify
- Additional arguments go in the middle
- Does **not** modify in place, creates a copy!

`732A94_AdvancedRHT2026_Lecture02_Slide34.R`

Functionals “apply family of functions”

Higher order functions, common in mathematics and functional languages.

Pros:

- Faster alternative to loops
- Easy to parallelize
- Encourages you to think about independence

Cons:

- Can’t handle serially dependent algorithms
- Can make code more difficult to read

Common Functionals

- `lapply()`
- `vapply()`
- `sapply()`
- `apply()`
- `tapply()`
- `mapply()`
- `rapply()` : recursive, used for nested lists
- `eapply()` : over elements of an environment

Note: use the `simplify` argument to get wanted output

outer() Apply function to pairs

```
outer(1:3,1:3)
```

```
##      [,1] [,2] [,3]
## [1,]    1    2    3
## [2,]    2    4    6
## [3,]    3    6    9
```

```
outer(c("a","b","c"), c("a","b","c"))
```

```
## Error in `tcrossprod()`:
## ! requires numeric/complex matrix/vector arguments
```

```
outer(c("a","b","c"), c("a","b","c"), paste, sep="-")
```

```
##      [,1]  [,2]  [,3]
## [1,] "a-a" "a-b" "a-c"
## [2,] "b-a" "b-b" "b-c"
## [3,] "c-a" "c-b" "c-c"
```

Common functionals `library(parallel)`

- `parLapply()`
- `parSapply()`
- `parApply()`
- `parRapply()` : row apply
- `parCapply()` : column apply
- `parLapplyLB()` : LB is load-balancing
- `parSapplyLB()` : LB is load-balancing

Note: use the `simplify` argument to get wanted output

Functional programming

- Programming paradigm
  - “How do I iterate over these values?”
  - “What function should be applied to each value?”
- Key thing is “the function”
  - Especially without side effects!

R is *not* purely functional, few languages are.

Anonymous functions

Functions without names

Often used within functionals

```
sapply(1:10, \(i){i^2}, simplify = TRUE)
```

```
##  [1]   1   4   9  16  25  36  49  64  81 100
```

```
sapply(1:10, function(i){i^2}, simplify = TRUE)
```

```
##  [1]   1   4   9  16  25  36  49  64  81 100
```

Closures: functions written by functions

From `732A94_AdvancedRHT2026_Lecture02_Slide44.R`

```
counter_factory<- function (){
  i<-0
  f<-function(){
    i<<-i+1
    i
  }
  f
}
f.c<-counter_factory()
s.c<-counter_factory()
```

Closures: functions written by functions

```
f.c()
```

```
## [1] 1
```

```
f.c()
```

```
## [1] 2
```

```
s.c()
```

```
## [1] 1
```

The functions have their own parent environment

Exception handling

Trap error instead of code crashing.

```
tryCatch({
  RCodeToRun
}, error = function(e){R code to handle error}
warning = function(w){R code to handle warning}
message = function(m){R code to handle message}
)
```

errors should be handled locally if possible or passed on.

To raise errors/warnings/messages use `stop()`, `warning()`, `message()`

R packages

An environment with functions and/or data

The way to share code and data

Number of packages on CRAN:

- 4 May 2018 roughly 12 500 packages
- 15 August 2022 – 18 428 packages
- 1 September 2026 – 24 801 packages

```
nrow(available.packages(available_packages_filters =
c("CRAN")))
```

R packages

To load a package use `library()`

Can call functions from installed packages using `packageName::functionName()` without loading.

Can use `:::` to access non-exported functions (bad practice, these are functions that the author don't want you to use).

Installation:

- `install.packages()` to install from CRAN
- `devtools::install_github()` to install from github
- `devtools::install_local()`

Package namespace

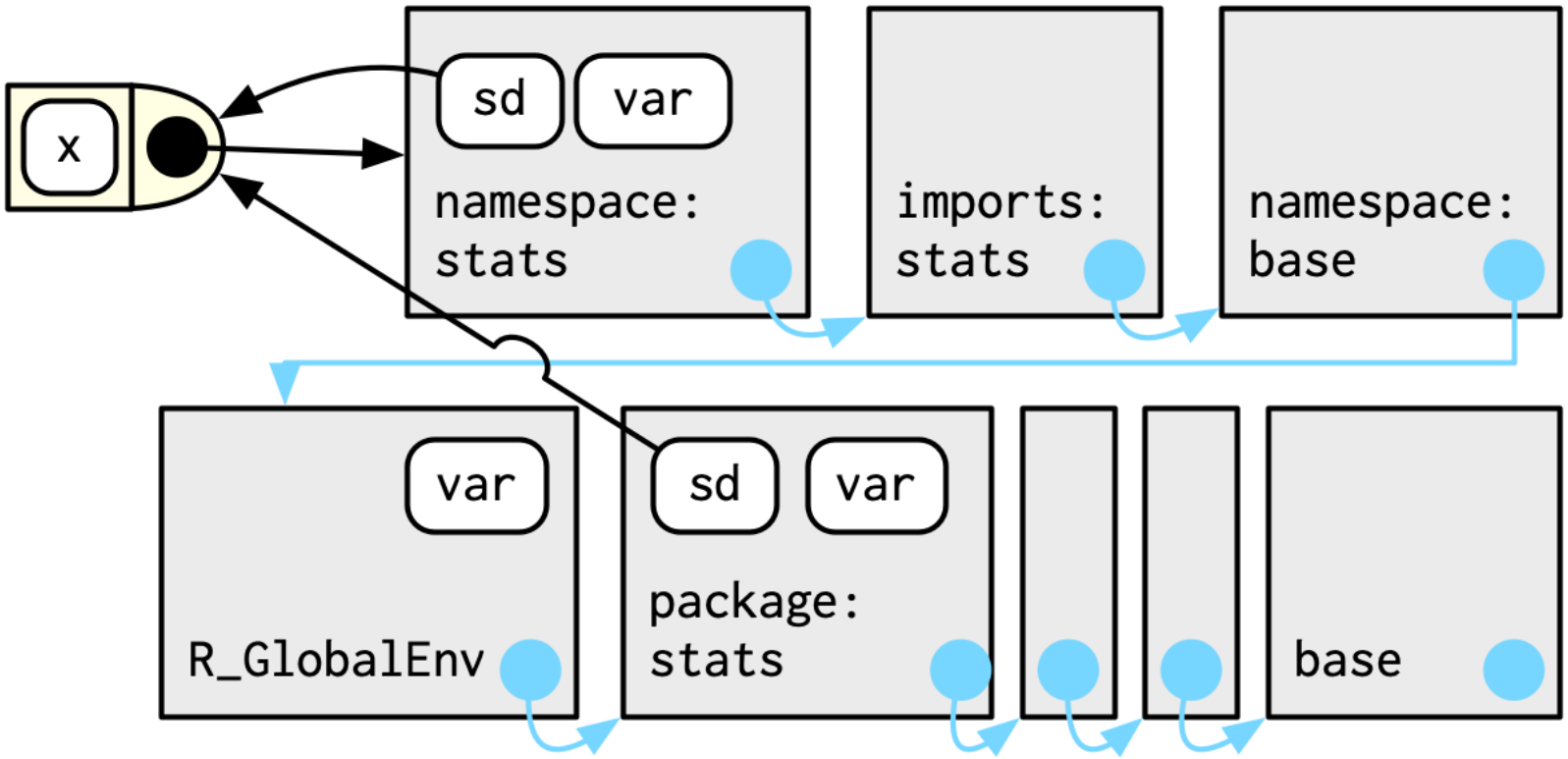

Figure 4: Package namespace (H. Wickham Adv. R)

Which are good packages?

Examine the package!

1. Who is the author?
2. When was it last updated?
3. Is it in development?

Version numbering `[major].[minor].[patch]`

Questions?